

# Övning 8: Cars – Full CRUD med TypeScript

Koppla ihop ditt C# Minimal API med en typad frontend i TypeScript

I den här övningen ska du koppla ihop ditt C# Minimal API med en frontend i HTML och **TypeScript**. Målet är att bygga en komplett applikation där du kan hämta, lägga till, uppdatera och ta bort bilar direkt från webbläsaren med full typsäkerhet!

## Mål med övningen

- Skriva typade och asynkrona anrop med `async`, `await` och `fetch`.
- Koppla frontend mot API:ets CRUD-endpoints (GET, POST, PUT, DELETE).
- Använda **Interfaces** för att typa dina datamodeller i frontenden.
- Hantera DOM-element i TypeScript genom att arbeta med korrekt typning och element-referenser.

## Förberedelser & Datamodell

1. Säkerställ att ditt C# Minimal API körs ( `dotnet run` ) och är tillgängligt via Swagger.
2. Skapa ett **Interface** i din TypeScript-kod som mappar exakt mot din C#-bilmodell (kom ihåg att ID kan vara valfritt ( `?` ) för nya bilar).

## Dina Uppgifter

### Nivå 1: Skapa ny bil (CREATE - POST)

Implementera funktionalitet för att spara en ny bil till servern via API:et.

**HTTP-metod:** POST | **Endpoint:** `/api/bilar`

- Lyssna på ditt bilformulärs submit-event.
- Hämta värdena från input-fälten. Se till att du refererar till och castar dina DOM-element till rätt typ så att du kan läsa av deras värden typ-säkert.
- Kom ihåg att årsmodellen behöver skickas som ett faktiskt tal ( `number` ) till API:et, inte en sträng.
- Skicka datan med `fetch` till din POST-endpoint. Sätt korrekta headers för JSON-kommunikation och skicka bilobjektet i anropets body.
- Efter ett lyckat API-anrop: Töm formulärets fält och ladda om billistan automatiskt.

### Nivå 2: Ta bort en bil (DELETE - DELETE)

Gör det möjligt för användaren att ta bort en specifik bil ur systemet.

**HTTP-metod:** DELETE | **Endpoint:** `/api/bilar/{id}`

- När du renderar din billista (GET), lägg till en raderingsknapp på varje bil.
- Skapa en asynkron funktion för att utföra raderingen. Funktionen behöver ta emot bilens unika ID och skicka detta i fetch-anropets URL.
- **Tänk på:** Om du arbetar i en miljö med ES-moduler (t.ex. Vite) ligger inte dina funktioner globalt på `window`-objektet. Fundera på hur du bäst kopplar dina eventlyssnare på raderingsknapparna utan att skriva inline HTML-event ( `onclick=""` ).
- När raderingen bekräftats av servern: Ladda om listan direkt på skärmen.

### Nivå 3: Redigera en bil (UPDATE - PUT)

Gör applikationen komplett genom att implementera uppdatering av befintliga bilar.

**HTTP-metod:** PUT | **Endpoint:** `/api/bilar/{id}`

- Lägg till en redigeringsknapp på varje bil i listan.
- När användaren klickar på redigera ska bilens nuvarande uppgifter populeras i formuläret så att användaren ser vad de ändrar.
- Du behöver hålla reda på \*vilket\* ID som just nu redigeras. Hitta ett sätt att spara detta tillfälligt i ditt TypeScript-skript.
- När formuläret sparas ska din kod nu avgöra om det är en ny bil som skapas (POST) eller en befintlig bil som uppdateras (PUT).
- Skicka ändringarna med `method: 'PUT'` till din redigerings-endpoint på API:et.

### Felsökningstips

- **Attribut i JSON vs C#:** Kom ihåg att C# Minimal API som standard kan returnera dina fält i `camelCase` (t.ex. `brand`) även om dina egenskaper i C# heter `Brand` (PascalCase). Se till att ditt interface matchar det som API:et faktiskt skickar ut.
- **Kompilatorfel i VS Code:** Om kompilatorn varnar för röda vågiga linjer på DOM-element, kom ihåg att tala om för TypeScript exakt vilken typ av element du hämtat. Det är sällan en bra idé att bara undertrycka felen med `any`!
- **Använd nätverksfliken:** Om du får felkoder som `400` eller `404` i konsolen, öppna webbläsarens utvecklarverktyg (F12) och granska nätverksanropet för att se exakt vad för JSON-data din TypeScript-kod skickar iväg.